TERRAFORM + LOCALSTACK IAM

INFRASTRUCTURE AS CODE

A-Z Practical Lab Guide

Terraform | LocalStack | AWS IAM | Least Privilege

IAM group	Admins-1
IAM user	CloudAdmin_Aliff
Managed policy	AdministratorAccess
Local endpoint	http://localhost:4566

Lab objective
Create an IAM group and user, attach AdministratorAccess to the group, add the user to the group, verify the result independently, prove idempotency, and clean up safely.

Prepared for: Aliff

Guide date: 13 August 2026

Workflow at a glance

1. INSTALL	2. CONFIGURE	3. PROVISION	4. VERIFY
Terraform AWS CLI Docker	LocalStack main.tf Credentials	init plan apply	AWS CLI no changes destroy

Required final state

Resource	Required value	Purpose
aws_iam_group	Admins-1	Central permission assignment
aws_iam_user	CloudAdmin_Aliff	Named IAM identity
group policy attachment	AdministratorAccess	Attach policy to group
user group membership	User -> Admins-1	User inherits group permission

Important security note
Never put the LocalStack Auth Token in main.tf, screenshots, reports, Git, or chat. If a token has been exposed, rotate it in the LocalStack Web Application.

Evidence checklist

- terraform init successfully initialized
- terraform validate reports a valid configuration
- terraform plan shows 4 to add
- terraform apply shows 4 added
- AWS CLI proves group membership and attached policy
- Second terraform plan reports No changes
- terraform destroy reports 4 destroyed

STEP 1 Install Terraform

Install Terraform from the HashiCorp APT repository on Kali Linux.

```
sudo apt update
sudo apt install -y wget gpg lsb-release
```

```
wget -O - https://apt.releases.hashicorp.com/gpg | \
sudo gpg --dearmor --yes \
  -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com
bookworm main" | \
sudo tee /etc/apt/sources.list.d/hashicorp.list
```

```
sudo apt update
sudo apt install -y terraform
terraform --version
```

Expected result
Terraform version information is displayed. An update notice is not an installation failure.

STEP 2 Check prerequisites

Confirm AWS CLI and Docker are available.

```
aws --version
docker --version
docker ps
```

The existing Kubernetes KIND container may continue running. It does not prevent this IAM lab as long as port 4566 is available.

STEP 3 Load the LocalStack Auth Token

Copy a Developer Auth Token from the LocalStack Web Application. Do not paste it into the command itself.

```
read -s "LOCALSTACK_AUTH_TOKEN?Paste token, then press Enter: "
echo
[[ -n "$LOCALSTACK_AUTH_TOKEN" ]] && echo "Token loaded" || echo "Token missing
```

Zsh command
Kali uses Zsh by default. The read syntax above keeps the token hidden while it is pasted.

STEP 4 Start LocalStack

Run the IAM and STS services on localhost port 4566.

If an old failed container exists, remove only that named container:

```
docker rm -f localstack
```

```
docker run -d \
  --name localstack \
  -p 4566:4566 \
  -e SERVICES=iam,sts \
  -e LOCALSTACK_AUTH_TOKEN="$LOCALSTACK_AUTH_TOKEN" \
  localstack/localstack
```

Remove the token from the current terminal session after Docker receives it:

```
unset LOCALSTACK_AUTH_TOKEN
```

```
docker ps --filter "name=localstack"
```

Expected result
The LocalStack container status should show Up and healthy. Port 4566 should be published.

STEP 5 Prepare the Terraform workspace

Keep the Terraform configuration in its own folder.

```
mkdir -p ~/terraform-iam-lab
mv ~/Downloads/main.tf ~/terraform-iam-lab/
cd ~/terraform-iam-lab
ls -l
```

If mv reports that the source file does not exist after it worked once, the file has already been moved. Confirm that main.tf appears in the destination folder.

STEP 6 Confirm the names in main.tf

The group and user names must match the verification commands.

```
grep -n 'name = "Admins-1"' main.tf
grep -n 'CloudAdmin_Aliff' main.tf
```

Required names
IAM group: Admins-1
IAM user: CloudAdmin_Aliff
Managed policy: AdministratorAccess

Complete main.tf configuration

This file declares all four required Terraform resources and points the AWS provider to LocalStack.

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
    }
  }
}

provider "aws" {
  access_key = "test"
  secret_key = "test"
  region = "us-east-1"

  endpoints {
    iam = "http://localhost:4566"
    sts = "http://localhost:4566"
  }

  skip_credentials_validation = true
  skip_metadata_api_check = true
  skip_requesting_account_id = true
}
```

```
resource "aws_iam_group" "admins" {
  name = "Admins-1"
}

resource "aws_iam_group_policy_attachment" "admin_policy" {
  group = aws_iam_group.admins.name
  policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess"
}

resource "aws_iam_user" "cloud_admin" {
  name = "CloudAdmin_Aliff"
}

resource "aws_iam_user_group_membership" "cloud_admin_membership" {
  user = aws_iam_user.cloud_admin.name

  groups = [
    aws_iam_group.admins.name
  ]
}
```

Permission design

The policy is attached to the group, not directly to the user. CloudAdmin_Aliff inherits the permission through membership in Admins-1.

STEP 7 Initialize Terraform

Download and initialize the AWS provider.

```
terraform init
```

Screenshot
Capture the message: Terraform has been successfully initialized!

STEP 8 Format and validate

Format main.tf and verify that the configuration is syntactically valid.

```
terraform fmt
terraform validate
```

Expected result
Success! The configuration is valid.

If the provider cache does not match the dependency lock file:

```
rm -r -- .terraform
rm -f -- .terraform.lock.hcl
terraform init -upgrade
terraform validate
```

STEP 9 Preview with terraform plan

Review every proposed change before provisioning resources.

```
terraform plan
```

Expected result
Plan: 4 to add, 0 to change, 0 to destroy.

STEP 10 Create the resources

Apply the reviewed plan to LocalStack.

```
terraform apply
# Enter a value: yes
```

Expected result
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

STEP 11 **Verify independently with AWS CLI**

Do not rely only on the Terraform Apply complete message.

Set dummy credentials for LocalStack:

```
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=us-east-1
```

Verification A - confirm group membership:

```
aws --endpoint-url=http://localhost:4566 iam get-group \
  --group-name Admins-1
```

Required evidence
The output must contain UserName: CloudAdmin_Aliff and GroupName: Admins-1.

Verification B - confirm the attached group policy:

```
aws --endpoint-url=http://localhost:4566 iam list-attached-group-policies \
  --group-name Admins-1
```

Required evidence
The output must contain PolicyName: AdministratorAccess and its AWS managed policy ARN.

STEP 12 **Prove idempotency**

Run the plan again after the infrastructure matches main.tf.

```
terraform plan
```

Expected result
No changes. Your infrastructure matches the configuration.

This demonstrates idempotency: running Terraform again does not create duplicate resources or make unnecessary changes.

STEP 13 Destroy the Terraform-managed resources

Final cleanup required by the lab PDF.

```
terraform destroy
# Enter a value: yes
```

Expected result
Destroy complete! Resources: 4 destroyed.

The destroy command removes only the four resources tracked in Terraform state. Keep main.tf and the screenshots for submission.

STEP 14 Stop and remove the LocalStack container

Perform this only after terraform destroy has completed successfully.

```
docker stop localstack
docker rm localstack
```

What remains
The Docker image and ~/terraform-iam-lab/main.tf remain available. Only the named LocalStack container is removed.

Submission checklist

#	Item	Evidence
1	main.tf	Four required resources and LocalStack provider endpoint
2	Initialize	Screenshot of successful terraform init
3	Validate	Screenshot showing configuration is valid
4	Plan	Screenshot showing 4 to add
5	Apply	Screenshot showing 4 added
6	Verification	Screenshots of group membership and attached policy
7	Idempotency	Screenshot showing No changes
8	Destroy	Screenshot showing 4 destroyed
9	Reflection	Short explanation of the IaC advantage

Short reflection

Submission-ready answer
Terraform is considered Infrastructure as Code because infrastructure resources are defined in a reusable and version-controlled configuration file. Compared with manual AWS CLI commands, Terraform provides consistent deployment, previews changes before applying them, tracks managed resources using state, and can safely recreate or destroy the same infrastructure.

Quick troubleshooting

Problem	Likely cause	Action
LocalStack exits with code 55	Auth Token missing, invalid, or expired	Create or rotate a Developer Auth Token, load it securely, then recreate the container.
curl cannot connect to port 4566	LocalStack stopped or unhealthy	Run docker ps -a and docker logs localstack.
Provider checksum mismatch	Cached provider and lock file differ	Remove .terraform and .terraform.lock.hcl, then run terraform init -upgrade.
terraform plan shows unexpected changes	main.tf and current resources differ	Review resource names, provider endpoint, and Terraform state before applying.
AWS CLI cannot find the group	Wrong endpoint or group name	Use endpoint http://localhost:4566 and group name Admins-1.

Final security action
Because an Auth Token was previously exposed, rotate it in the LocalStack Web Application. Do not reuse the exposed token.

End of guide

Keep main.tf, the Terraform lock file, and all required screenshots together in your submission folder.